



UACJ

UNIVERSIDAD AUTÓNOMA DE CIUDAD
JUÁREZ

EXTENSION MULTIDISCIPLINARIA EN CIUDAD
UNIVERSITARIA

Control Brazo Robotico de 6 DOF

Autor: César Jared Andrade Ramirez

Docente: Diana Yazel Ortiz Muñoz

Curso: Redes Industriales

Grupo: C-L1

Fecha: Mayo 19 2026

1 Introduccion

El proyecto desarrolla un sistema de operación inalámbrica para un brazo robótico de 6 DOF, mediante una arquitectura distribuida basada en redes Wi-Fi y la conexiones TCP/IP.

Alineado con los principios de la Industria 4.0, el sistema garantiza un control casi milimétrico y en tiempo real mediante la siguiente estructura:

- Interacción Cliente-Servidor: Los comandos vectoriales se envían desde un teléfono celular mediante el protocolo HTTP utilizando estructuras JSON.
- Enrutamiento y Aislamiento: Una Raspberry Pi actúa como servidor intermedio, aislando los procesos de red y comunicándose con el hardware a través de túneles de Sockets TCP de baja latencia.
- Administración Remota: El despliegue y depuración del sistema se gestiona de forma cifrada mediante SSH, operando el equipo en modalidad "Headless" (sin monitor ni teclado).

Esta simbiosis de protocolos establece un marco de trabajo escalable que permite controlar físicamente los actuadores desde cualquier punto dentro de la cobertura de la red local.

2 Desarrollo

El desarrollo del proyecto se enfocó en la transición de un modelo geométrico abstracto hacia un sistema de control directo por velocidad angular distribuido en una arquitectura de tres capas.

2.0.1 Que se hizo

- Migración del control: Se reemplazó un sistema inicial basado en cinemática inversa cartesiana (XYZ)—el cual provocaba extensiones forzadas peligrosas en los eslabones debido a las restricciones de orientación—por un sistema de control de velocidad angular en tiempo real mediante joysticks virtuales interactivos.
- Interfaz Táctil Adaptable y Simulación 3D: Se desarrolló una aplicación web responsiva con scroll vertical libre que integra la librería Nipple.js para la captura de movimientos fluidos y Three.js para la renderización tridimensional amortiguada del estado del robot, permitiendo operar el sistema cómodamente desde un teléfono celular.
- Automatización y Poses de Trabajo: Se programó un secuenciador de trayectorias capaz de almacenar los ángulos reales de los seis motores en un sistema de memoria, permitiendo la reproducción de rutinas en modalidades de un solo ciclo o bucle continuo. Adicionalmente, se configuró un panel de macros para posiciones preestablecidas, aunque solo se colocaron 2 al final.

2.0.2 Como se logro

Fue necesario el uso de tres entornos (frontend, backend, hardware) para que el proyecto funcionara segun lo establecido

- Capa de Cliente (Frontend): En el navegador del dispositivo móvil, un bucle de control de velocidad (Flight Loop) programado en JavaScript se ejecuta a una frecuencia de 10 Hz (cada 100 ms). Este ciclo traduce de manera continua el desplazamiento del joystick en incrementos angulares y los acota localmente entre 0° y 180°. Para evitar el colapso del servidor por saturación de paquetes, se implementó una válvula de concurrencia asíncrona mediante un candado lógico que retiene los envíos hasta recibir la confirmación de la petición anterior. Los datos consolidados se transmiten como objetos JSON vía peticiones HTTP POST de corto alcance.
- Capa de Servidor (Backend): Una Raspberry Pi 4 ejecuta un servidor web bajo el micro-framework Flask. Al recibir la petición en la ruta correspondiente, el backend realiza una verificación estricta de seguridad en el servidor para asegurar que los datos no estén corruptos y cumplan con los límites físicos de los motores. Tras la validación, Flask abre de forma interna un Socket TCP local para canalizar la información hacia un script de bajo nivel, el cual serializa el arreglo numérico en una cadena de texto simple separada por comas y finalizada con un salto de línea (por ejemplo: S:90,45,180,0,135,90).
- Capa de Control (Hardware): La trama de texto se inyecta mediante un enlace físico USB (protocolo Serial a 115200 baudios) hacia un microcontrolador Arduino. El firmware del Arduino almacena los caracteres en un buffer intermedio y, al detectar el carácter de terminación, fragmenta la cadena para recuperar de forma exacta los valores enteros. Estos enteros se convierten directamente en señales de modulación por ancho de pulso (PWM) que comandan la posición de las bobinas de los servomotores físicos. Finalmente, un lazo secundario de telemetría analógica extrae el consumo de corriente de los sensores del robot y lo devuelve en sentido inverso hasta la pantalla del celular para el monitoreo de esfuerzo.

2.0.3 Como funciona

Aqui presentaremos el paso a paso de lo que sucede cuando el operador presiona el boton de agarrar de la pinza del endeffector

1. Acción realizada por el usuario: El operador presiona el botón interactivo con la etiqueta "AGARRAR" en la pantalla táctil de su teléfono celular.
2. Fenómeno en la página HTML e interfaz del cliente: La pulsación activa de forma inmediata un evento dinámico programado en JavaScript que reescribe el arreglo interno de variables angulares del end effector, fijándolo en el valor predeterminado de sujeción (90 grados). El hilo de renderizado

de Three.js actualiza instantáneamente el modelo digital en pantalla. Por último, la API Fetch empaqueta el estado completo de los seis motores en formato JSON e inicia una petición asíncrona mediante el método HTTP POST.

3. Recepción en el servidor web Flask: El servidor Flask que se ejecuta en la Raspberry Pi captura la petición HTTP POST entrante bajo la ruta reservada `/set_servos`. El backend realiza la decodificación del formato JSON, extrayendo las variables numéricas asociadas a cada identificador de motor.
4. Despacho del comando a través del transporte de red interna: Una vez completado el filtro de validación, el hilo de Flask abre un Socket de red de bucle local (Loopback TCP) apuntando a la dirección IP interna y al puerto asignado para la comunicación del hardware de bajo nivel, enviando los valores limpios en una cadena lineal.
5. Procesamiento del servidor TCP intermedio: El script de control de bajo nivel que escucha en el puerto de red interna intercepta la cadena de datos binarios y la decodifica bajo la norma UTF-8, consolidando una trama de texto normalizada que incluye un identificador de comando y un terminador de línea, por ejemplo: `'S:90,45,180,0,135,90 '`
6. Comando inyectado al microcontrolador Arduino: La trama de texto estructurada se transmite por el cable USB a través del bus de comunicación Serial a una velocidad configurada de 115200 baudios. El firmware del Arduino almacena secuencialmente los caracteres entrantes en su memoria intermedia (buffer) hasta identificar el salto de línea (`' '`), momento en el cual segmenta el texto usando las comas como delimitadores para recuperar de forma exacta el entero correspondiente al motor final
7. Respuesta dinámica del actuador físico: El microcontrolador procesa el valor numérico y altera los registros de temporización internos de sus pines digitales de salida, modificando el ancho de pulso de la señal PWM (Modulación por Ancho de Pulso). El circuito integrado dentro del servomotor decodifica esta señal, alimenta el motor de corriente continua y desplaza el tren de engranajes mecánicos hasta alcanzar la posición correspondiente a los 90 grados, ejecutando el cierre físico de la garra
8. Retroalimentación devuelta hacia el operador: El servidor Flask retorna un código de estado HTTP 200 OK hacia el navegador móvil. De forma paralela, el lazo asíncrono de telemetría extrae la lectura del sensor de corriente analógico en el hardware; esta medición es devuelta en sentido inverso a través de la red y actualiza el indicador numérico de amperaje en el teléfono celular, confirmando visualmente el esfuerzo real ejercido por el actuador

3 Partes fundamentales del código

- Bucle de "Flight Loop" (JavaScript): Lo que hice fue tomar la entrada de los joysticks virtuales y programarla para que se traduzca en cambios fluidos de velocidad angular. Además, me aseguré de aplicar límites físicos de seguridad directamente en el código antes de enviar cualquier información

```
setInterval(() => {
  let huboCambio = false;
  for(let i=0; i<4; i++) {
    if(Math.abs(joystickVels[i]) > 0.1) {
      let nuevoAngulo = targetAngles[i] + joystickVels[i];
      nuevoAngulo = Math.max(0, Math.min(180, nuevoAngulo));
      if(Math.round(nuevoAngulo) !== targetAngles[i]) {
        targetAngles[i] = Math.round(nuevoAngulo);
        huboCambio = true;
      }
    }
  }
  if(huboCambio) {
    actualizarTextos();
    sendAngl
```

- Válvula de Red con Candado logico (JavaScript): Garantiza estabilidad en la red inalámbrica mediante el uso de una bandera de estado (candado lógico). Esto impide que las peticiones consecutivas saturen la memoria del servidor.

```
let envioEnCurso = false;
async function sendAngles() {
  if(envioEnCurso) return;
  envioEnCurso = true;
  const payload = {};
  targetAngles.forEach((ang, idx) => { payload['s${idx+1}'] = ang; });
  try {
    await fetch('/set_servos', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload)
    });
  } catch (e) {}
  envioEnCurso = false;
}
```

- Filtro de Autenticación y Validación en Servidor (Python / Flask): Establece la barrera de seguridad en el backend. Valida de manera estricta que la petición provenga de una sesión autorizada y somete los datos numéricos entrantes a una comprobación de límites operativos para blindar el hardware contra datos corruptos.

```

@app.post("/set_servos")
def set_servos():
    if not is_logged_in():
        return jsonify({"ok": False, "error": "No autorizado"}), 401
    data = request.get_json(silent=True) or {}
    try:
        default_angles = [90, 45, 180, 0, 135, 90]
        servos = [int(data.get(f"s{i}", default_angles[i-1])) for i in range(1, 7)]
        if not all(0 <= s <= 180 for s in servos):
            raise ValueError
    except:
        return jsonify({"ok": False, "error": "Angulos invalidos (0-180)"}), 400

```

- Serialización de Comandos e Inyección por Sockets (Python): Transforma los datos validados del ecosistema web en una trama compacta formateada con delimitadores, abriendo un túnel de socket TCP para transferir la información hacia las capas físicas de control.

```

cmd = f"S:{servos[0]},{servos[1]},{servos[2]},{servos[3]},{servos[4]},{servos[5]}"
resp = send_cmd(cmd)
return jsonify({"ok": not resp.startswith("ERR"), "cmd": cmd, "resp": resp})
def send_cmd(cmd: str) -> str:
    with socket.create_connection((TCP_HOST, TCP_PORT), timeout=3) as s:
        s.sendall((cmd + "\n").encode("utf-8"))
        return s.recv(1024).decode("utf-8")

```

- Dinamica del modelo digital (Three.js): Ejecuta cálculos de interpolación lineal (Lerp) para aproximar progresivamente la postura del modelo tridimensional virtual a las variables físicas reales del brazo, eliminando transiciones abruptas en pantalla.

```

function animate() {
    requestAnimationFrame(animate);
    for(let i = 0; i < 6; i++) {
        visualAngles[i] += (targetAngles[i] - visualAngles[i]) * 0.1;
        updateRobotVisual(i, visualAngles[i]);
    }
}

```

```
        renderer.render(scene, camera);  
    }  
    animate();
```

4 Conclusion

4.0.1 Que ventajas tiene integrar redes, programacion web y dispositivos electronicos en un proyecto mecatronico orientado a IoT?

Integrar redes, programacion y dispositivos a un proyecto puede cambiar un mecanismo simple como puede ser simples eslabones, juntas y motores separados, en un sistema moderno. En este caso permitio un control universal wireless, pues desde cualquier dispositivo conectado a la red establecida en la raspberry se puede acceder (con usuario y contraseña establecidos) y controlar de forma remota el robot. Permite un monitoreo en tiempo real de los componentes, permite saber su temperatura, consumo, angulo, etc, con las configuraciones pertinentes claro esta. Ademas permite crear sistemas faciles de actualizar y por tanto mejorables y escalables permitiendo asi una incersion en la industria.